Day-1

System Design

→ System design is the process of defining the architecture, interfaces and data for a system that satisfies specific requirements.

→ System design meets the needs of your business (or) organization through coherent and efficient systems. It requires a systematic approach to building any Engineering systems.

→ A good system design requires us to think about everything, from infrastructure all the way down to the data & how it's stored.

→ It answers questions like
↳ what components will the system have?
↳ How will users, servers, databases & API's interact?
↳ How will system handle many user's, failures & data storage.
↳ ~~(How will the system handle many user's, failures & data storage)~~
↳ How can it be fast, reliable & easy to maintain.

Eg: URL SHORTENER

→ requirements:
↳ Enter Long URL
↳ Get short URL
↳ Open the short URL & be redirected to the original one.

→ High Level design:

① Frontend: A webpage where users paste a long URL.
② Backend: Recieves Long URL, Generates a short code
③ Database: Short code → Original URL (stored)
④ Redirect services: when someone visits short.ly/abc123, the system looks up the original URL & redirects the user.

Scanned with CamScanner

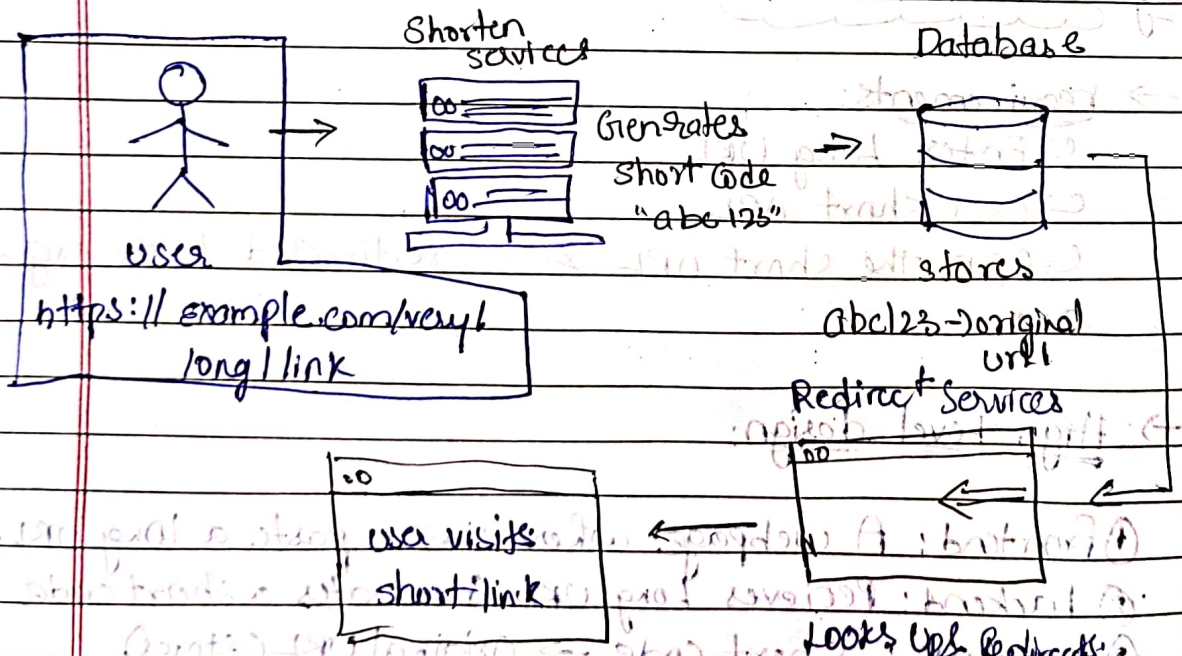
Date
Page No.

Flow:

→ User submits → https://example.com/very/long/link
→ Backend creates abc123
→ Database stores:
abc123 → long URL
→ User shares short.ly/abc123
→ when opened, system fetches the original URL & redirects.

→ Why system design matters:
↳ without system design
→ the app may become slow
→ data may become slow
→ Scaling to more user becomes difficult
→ failures can bring the whole system down.
↳ with good system design

↳ Systems are organized
↳ easier to scale
↳ easier to maintain
↳ better performance & reliability.



Short.ly/abc123
Redirects to Original URL.

FABER-CASTELL
Date
Page No.

Python Sample:

```
Class URLShortener:
    def __init__(self):
        self.url_map = {}
        self.counter = 1
    def shorten_url(self, long_url):
        short_code = "U" + str(self.counter)
        self.url_map[short_code] = long_url
        self.counter += 1
        return "https://short.ly/" + short_code.
    def redirect_url(self, short_url):
        short_code = short_url.split("/")[-1]
        return self.url_map.get(short_code, "URL not found")

shortener = URLShortener()
long_url = "https://example.com/very/long/link"
short_url = shortener.shorten_url(long_url)
print(long_url)
print(short_url)
print(shortener.redirect_url(short_url))
```

Java:

```
import java.util.HashMap;
import java.util.Map;
class URLShortener {
    private Map<String, String> urlMap;
    Private int Counter;
    Public URLShortener(){
        urlMap = new HashMap<>();
        counter = 1;
    }
    Public string shortenURL(string longURL){
        String shortcode = "U" + counter;
        Url.Map.put(shortcode, longURL);
        Count++;
        return "https://short.ly/"+shortcode
    }
    Public string redirectURL(string shortURL){
        string[] parts = shortURL.split("/");
        String shortcode = parts[parts.length-1]
        return.getOrDefault(Shortcode,
                "URL not found");
    }
```

Scanned with CamScanner